TEKSTREAM SOLUTIONS

AI/ML Architect

Technical Assessment

Cybersecurity Intelligence Platform

CONFIDENTIAL

This document contains proprietary information belonging to TekStream Solutions.

Do not distribute without written authorization.

Overview

This assessment evaluates your ability to design and build core ML components for a cybersecurity intelligence platform. You will work with a synthetic security alert dataset and implement a functioning prediction pipeline.

Parameters

Time Budget	4 hours maximum. We value pragmatic tradeoffs over perfection.
Language	Python. Any libraries are acceptable; justify your selections.
AI Tools	LLMs may be used as a development aid (Claude, Copilot, etc.), but architecture decisions, feature engineering, and mathematical reasoning must be yours. We will discuss your work in a follow-up conversation.
Partial Work	Partial submissions are acceptable. A well-reasoned approach to 2 tasks is better than a rushed attempt at all 3.
Deadline	5 business days from receipt.
Questions	Email hardik.desai@tekstream.com. Clarifying questions are encouraged and will not count against you.

Context

You are building the ML foundation for a Managed Detection and Response (MDR) platform. SOC (Security Operations Center) analysts receive a stream of security alerts from multiple detection tools. Each alert has properties like source, severity, type, timestamp, and asset context. Alerts are mapped to the MITRE ATT&CK framework, which classifies adversary behavior into tactics (the why) and techniques (the how). Analysts triage each alert and assign a disposition: true positive (real threat), false positive (noise), or escalated (needs deeper investigation).

A key design principle: risk is multi-dimensional. Vendor severity is a starting point, but true risk depends on at least three independent axes:

- **Inherent threat severity** — The danger of the threat type itself. Credential access and lateral movement are inherently more dangerous than reconnaissance or discovery, regardless of where they occur.
- **Asset/identity context** — The same technique on a domain controller vs. a student lab workstation carries different organizational risk.
- **Behavioral context** — Alert volume, timing, IOC correlation, and whether the activity fits a pattern of campaign progression across the kill chain.

Your models will operate under governance constraints:

- Thresholds must be configurable externally (not hardcoded in model code)
- Every prediction must be explainable (why this score?)
- Confidence must be calibrated (a score of 0.9 must mean 90% accuracy at that threshold)

Task 1: Data Generation and Feature Engineering

Required | Suggested time: 60 minutes

1A: Generate a Synthetic Alert Dataset

Create a Python script (generate_dataset.py) that produces a CSV of 10,000 synthetic security alerts with realistic statistical properties. The dataset must include the following columns:

Column	Description	Distribution Guidance
alert_id	Unique identifier	Sequential UUID
timestamp	When the alert fired	Span 90 days. FPs cluster during business hours; TPs are more uniform.
source_rule	Detection rule that fired	50 distinct rules. Some rules have >80% FP rate, others <10%.
mitre_tactic	MITRE ATT&CK tactic (the adversary objective)	Use the tactic reference table below. Reconnaissance and Discovery are most common; Impact and Exfiltration are rare.
mitre_technique	MITRE ATT&CK technique ID (e.g., T1059, T1078)	3–5 techniques per tactic. Each technique has a characteristic FP rate.
inherent_severity	Threat-intrinsic severity (1–4) based on tactic position in the kill chain	Derived from mitre_tactic. See reference table. Independent of vendor_severity.
vendor_severity	Vendor-assigned severity (1–5)	Severity 3 is most common; severity 5 is rare but mostly TP. Often disagrees with inherent_severity.
asset_type	Type of asset	server, workstation, iot, cloud_vm, domain_controller. DCs are rare but high-value.
asset_criticality	Business criticality (1–10)	Correlates with asset_type.
alert_count_1h	Alerts from same source in prior hour	FPs tend to burst; TPs are more isolated.
alert_count_24h	Alerts from same source in prior 24h	Same pattern, wider window.
has_ioc_match	Matches a known IOC (True/False)	TPs match IOCs ~40% of the time; FPs ~3%.
user_privilege_level	Privilege level of associated user	standard, admin, service_account. Service accounts generate more FPs.
time_since_last_alert	Seconds since last alert from source	Bursty FPs have short intervals.
disposition	Analyst decision	15% true_positive, 75% false_positive, 10% escalated.

MITRE ATT&CK Tactic Reference

Use this mapping to assign inherent_severity. The severity reflects where a tactic sits in the kill chain — later-stage tactics represent greater organizational danger regardless of the specific asset involved.

Tactic	Inherent Severity	Rationale

Reconnaissance	1 — Low	Information gathering. High volume, rarely actionable alone.
Resource Development	1 — Low	Adversary infrastructure setup. Typically observed externally.
Initial Access	2 — Medium	Entry point compromise. Serious but may be blocked downstream.
Execution	2 — Medium	Running adversary code. Dangerous but containable if caught early.
Persistence	2 — Medium	Maintaining foothold. Indicates intent to stay but not yet progressed.
Privilege Escalation	3 — High	Gaining elevated permissions. Amplifies all subsequent actions.
Defense Evasion	3 — High	Actively hiding. Sophisticated adversary behavior.
Credential Access	3 — High	Stealing credentials. Enables lateral movement and identity abuse.
Discovery	1 — Low	Internal reconnaissance. Noisy, high FP rate from legitimate tools.
Lateral Movement	4 — Critical	Spreading through environment. Active campaign progression.
Collection	3 — High	Gathering target data. Pre-exfiltration staging.
Command and Control	3 — High	Active adversary communication channel. Confirmed compromise.
Exfiltration	4 — Critical	Data leaving the environment. Damage may already be occurring.
Impact	4 — Critical	Destruction or disruption. Active harm in progress.

Note: vendor_severity and inherent_severity will frequently disagree. A vendor may score a Discovery alert as severity 4 because the signature is broadly flagged, while its inherent threat severity is only 1. Conversely, a subtle Credential Access technique might receive a low vendor severity but carries inherent severity 3. How candidates handle this disagreement is a key signal.

Requirements

- The feature correlations must be realistic and non-trivial. A competent data scientist should look at the pairwise statistics and say these patterns make sense for SOC alert data.
- Include at least two interaction effects (features that matter in combination but not alone). Document what they are and why they are realistic.
- Write a brief statistical summary showing dataset properties: class balance, key correlations, and interaction effects.

1B: Engineer 3 Derived Features

From the raw columns above, engineer 3 additional features that you believe would improve false positive prediction. Consider features that capture interactions across the three risk axes (inherent threat severity, asset/identity context, behavioral context). For each derived feature, provide:

1. Name and formula
2. Mathematical justification (why this feature carries signal)
3. Expected correlation with disposition

Evaluation Criteria for Task 1

Criterion	Points	What Distinguishes Great from Good
Realistic distributions and correlations	10	MITRE tactic distributions are realistic; vendor vs. inherent severity disagreements are modeled thoughtfully; interaction effects are mechanistically motivated.
Derived feature quality	8	Features capture cross-axis interactions (threat severity × asset context × behavior); mathematical justification is clear.
Code quality and reproducibility	7	Seeded randomness, configurable parameters, clean structure.

Task 2: False Positive Predictor

Required | Suggested time: 90 minutes

Build a model that predicts whether a security alert is a false positive. Your model must reason across all three risk dimensions: the inherent danger of the threat type (informed by MITRE tactic), the criticality of the targeted asset, and the behavioral context of the alert. A Credential Access technique on a domain controller with an IOC match is a fundamentally different signal than a Discovery alert bursting on an IoT sensor — even if the vendor scores them identically.

2A: Model Implementation

Create `fp_predictor.py` containing:

1. Data loading and preprocessing pipeline
2. Model training with proper train/validation/test splits
3. At least two model approaches compared (e.g., gradient boosting vs. logistic regression). Justify why you selected these two.
4. Hyperparameter tuning: a grid search is fine, Bayesian is better, a justified manual selection with reasoning is also acceptable.

2B: Calibrated Confidence Scores

The model must output calibrated probabilities, not just class labels. A prediction of P(false_positive) = 0.85 must mean that roughly 85% of alerts scored at that level are actually false positives.

1. Implement calibration (Platt scaling, isotonic regression, or another method)
2. Produce a reliability diagram (calibration curve) showing pre- and post-calibration performance
3. Briefly explain why calibration matters in a SOC context where analysts will see these scores

2C: Externalized Thresholds

The decision boundary must not be hardcoded. Implement a mechanism where thresholds are loaded from an external configuration file (YAML, JSON, or .env):

```
fp_threshold_auto_close: 0.95    # Alerts above this are auto-dismissed
fp_threshold_flag_review: 0.70    # Alerts between this and auto_close get quick review
# Alerts below flag_review go to full analyst triage
```

Show how changing these thresholds changes the operating characteristics: false positive pass-through rate, missed true positive rate, and analyst workload.

2D: Evaluation Report

Produce an evaluation report (`evaluation.md` or a notebook section) containing:

1. Precision, recall, and F1 at your chosen operating point, and why you chose that point
2. ROC curve and AUC for both models
3. Feature importance analysis: which features drive predictions? How does inherent_severity compare to vendor_severity in importance?

4. Error analysis (general): what do the misclassified alerts look like? Are errors concentrated in specific MITRE tactics?
5. Severity conflict analysis (see below)
6. A 2–3 paragraph discussion: *If I were deploying this to a SOC with 500 alerts/day, here is what I would tell the analysts about how to interpret and trust these scores.*

Severity Conflict Analysis (Required)

This is the model's hardest test. Isolate the subset of alerts where vendor_severity and inherent_severity maximally disagree:

- **High vendor / low inherent:** Vendor severity 4–5 but inherent severity 1 (e.g., a Discovery alert that a vendor scores as critical).
- **Low vendor / high inherent:** Vendor severity 1–2 but inherent severity 3–4 (e.g., a subtle Credential Access or Lateral Movement technique that a vendor under-scores).

For each conflict subset, report:

- Precision and recall on that subset specifically (not the overall test set)
- Which signal the model actually learned to trust when the two severity scores disagree. Does it follow vendor_severity, inherent_severity, or something else entirely?
- What this means operationally. If the model collapses to vendor_severity when the two signals conflict, it replicates the exact bias that experienced analysts learn to override. If it learned to weight inherent_severity or contextual features, explain what enabled that.

> *Why this matters: The platform we are building exists precisely to correct for vendor severity biases. A model that auto-closes a Lateral Movement alert because the vendor scored it low, or escalates a Discovery alert because the vendor scored it high, is worse than no model at all. We need to see that you understand this failure mode and can diagnose whether your model falls into it.*

Evaluation Criteria for Task 2

Criterion	Points	What Distinguishes Great from Good
Model selection and comparison	10	Reasoning about inductive biases; model handles the multi-dimensional nature of risk (threat type × asset × behavior) thoughtfully.
Calibration implementation	10	Reliability diagram shows meaningful improvement; discussion of why it matters in SOC context.
Externalized thresholds	8	Threshold impact analysis shows understanding of precision-recall tradeoff across operating points.
Evaluation depth	12	Severity conflict analysis reveals whether the model learned the right signal; deployment discussion is specific and actionable.
Code quality	10	Clean separation of concerns, reproducible, well-documented.

Task 3: Dynamic Risk State Estimation

Stretch Goal | Suggested time: 60 minutes

> *This task is optional. Attempt it only if you complete Tasks 1 and 2 with time remaining. A thoughtful partial attempt is more valuable than a rushed complete one.*

The Problem

Individual alert predictions are useful, but a SOC needs to understand the risk state of a customer environment over time. Risk is not static — it evolves as alerts accumulate, incidents are resolved, and the threat landscape shifts.

Critically, not all alerts contribute equally to risk. An alert mapped to Lateral Movement (inherent severity 4) should move the risk estimate more than a Discovery alert (inherent severity 1), even if both are confirmed true positives. And a sequence of alerts progressing through the MITRE kill chain — Initial Access followed by Execution followed by Credential Access — is qualitatively more dangerous than three unrelated alerts at the same severity levels.

Design and implement a risk state estimator that maintains a rolling estimate of environment risk, weighted by inherent threat severity and sensitive to kill chain progression.

3A: Mathematical Formulation

Write a brief document (`risk_model.md`) describing your approach. You may choose any framework. Some possibilities:

1. **Bayesian state estimation:** Prior belief about risk, updated by each alert.
2. **Hidden Markov Model:** Latent risk states (nominal, elevated, critical) with observable alert emissions.
3. **Exponentially weighted moving average:** With alert-severity weighting and configurable decay.
4. **Something else entirely:** Justify it.

Your formulation must address:

1. How does a new alert update the risk estimate? How is the update weighted by inherent_severity vs. vendor_severity?
2. How does the passage of time without alerts decrease the risk estimate (risk decay)?
3. How do you handle multi-scale risk (a single critical alert vs. a slow accumulation of medium alerts)?
4. Bonus: How would you detect kill chain progression — a correlated sequence of alerts moving from early-stage tactics (Initial Access) to late-stage tactics (Lateral Movement, Exfiltration)?

3B: Implementation

Create `risk_estimator.py` that:

- Consumes a time-ordered stream of alerts from your Task 1 dataset
- Maintains a risk state per asset_type or per simulated customer
- Produces a risk score (0–1) at any point in time
- Demonstrates risk increasing during alert bursts and decaying during quiet periods

3C: Visualization

Produce a time-series plot showing the risk score over 90 days for one simulated customer, with actual alert events overlaid (colored by disposition). Annotate at least one period where the estimator correctly identifies elevated risk, and one where it produces a false alarm.

Evaluation Criteria for Task 3

Criterion	Points	What Distinguishes Great from Good
Mathematical formulation	10	Principled framework with clear update and decay mechanics; inherent severity weighting is not ad hoc.
Working implementation	8	Time-series plot is sensible; risk responds more strongly to high-inherent-severity alerts.
Systems thinking	7	Formulation accounts for kill chain progression and could integrate with a larger governed platform.

Deliverable Structure

```
assessment/
    README.md                       # Setup, how to run, summary of approach
    requirements.txt                 # Dependencies
    policy.yaml                      # External threshold configuration
    generate_dataset.py              # Task 1A
    fp_predictor.py                  # Task 2A-C
    evaluation.md                    # Task 2D
    risk_model.md                    # Task 3A (if attempted)
    risk_estimator.py                # Task 3B (if attempted)
    outputs/
        alerts_dataset.csv              # Generated dataset
        calibration_curve.png           # Task 2B
        roc_curves.png                  # Task 2D
        feature_importance.png           # Task 2D
        threshold_analysis.png           # Task 2C
        risk_timeline.png               # Task 3C (if attempted)
```

Submit as a private GitHub repository (shared with evaluators) or a ZIP archive to hardik.desai@tekstream.com.

Scoring Summary

Component	Points	Status
Task 1: Data Generation + Features	25	Required
Task 2: False Positive Predictor	50	Required
Task 3: Dynamic Risk State Estimation	25	Stretch Goal
Total	**100**	

Interpretation

Score	Interpretation
70+	Strong builder. Proceed to offer discussion.
50–69	Capable builder with gaps. Discuss specific areas in follow-up conversation.
30–49	Primarily architect. May need implementation support pairing.
<30	Does not meet builder bar for this role.

Follow-Up Conversation

After submission, we will schedule a 30-minute code review conversation where you walk us through your decisions. We are specifically interested in:

- Why you chose your model architectures and what tradeoffs you considered

- What your severity conflict analysis revealed — which signal did your model learn to trust, and if you could retrain with that insight, what would you change?
- What you would do differently with more time, more data, or access to historical analyst decisions
- How your risk state estimator (if attempted) would integrate with a governance layer that enforces policy-driven thresholds and records all state changes
- How you would adapt this work for a customer onboarding scenario where you have zero historical data for that specific customer

> *This conversation matters as much as the code. We want to hear you think.*